Reports 2–9

Развернутый исследовательский разбор серии экспериментов по графовой памяти RIN

Рабочая ChatGPT-версия исследовательского свода

18 августа 2026 г.

Этот проект собирает в одном месте исследовательскую линию от `exp2` до `exp9a`. Жанр текста – не набор коротких отчетов, а последовательный курс-разбор: что именно проверял каждый эксперимент, почему он возник именно в этот момент, какую альтернативу он пытался отличить от конкурирующей гипотезы, какой стенд был собран, что показали результаты и как они изменили дальнейший план работ. Отдельно проговариваются пределы каждого результата: где мы имеем дело с полноценным эмпирическим опровержением, где с диагностикой, а где лишь с хорошо поставленной развилкой для следующего шага.

Оглавление

Предисловие **1**

1 Исследовательская программа: граф, сигнал, метрики **5**

1.1 Почему серия вообще начинается с эмбриона 5

1.2 Как выглядит базовый графовый объект 5

1.3 Как выглядит retrieval в базовой версии 6

1.4 Три уровня retrieval-скоринга 7

1.4.1 Локальный транспорт 7

1.4.2 Локальное удержание 7

1.4.3 Глобальная структурная слава 7

1.5 Роль embedding-пространства 8

1.6 Какими метриками пользовалась серия 8

1.7 Почему понадобились золотые миры 9

1.8 Как устроен дальнейший рассказ 9

2 От эмбриона к золотому миру: `exp2` и `exp3` **11**

2.1 `exp2`: эмбриональный граф из реальных источников 11

2.1.1 Смысл эксперимента 11

2.1.2 Стенд 11

2.1.3 Что здесь могло получиться 12

2.1.4 Что реально произошло 12

2.1.5 Инфраструктурный результат 13

2.1.6 Что эксперимент оставил открытым 13

2.2 `exp3`: золотой микромир как измеритель retrieval 13

2.2.1 Зачем понадобился второй стенд 13

2.2.2 Конструкция золотого мира 13

2.2.3 Какие именно сценарии проверялись 14

2.2.4 Главный результат 14

2.2.5 Что именно это доказало 14

2.2.6 Связь `exp2` и `exp3` 15

3 От семантического gate к write-path: exp4 и exp5 **17**
3.1 exp4: semantic gate как первая попытка исправить retrieval 17
3.1.1 Что именно проверялось 17
3.1.2 Постановка 17
3.1.3 Что ожидалось при двух конкурирующих картинах 17
3.1.4 Результат 17
3.1.5 Почему это особенно важный результат 18
3.1.6 Какой следующий вопрос возник немедленно 18
3.2 exp5: write-path как отдельный архитектурный срез 18
3.2.1 Почему эта глава здесь вообще стоит 18
3.2.2 Постановка 19
3.2.3 Какая альтернатива тут была на самом деле 19
3.2.4 Что обнаружилось 19
3.2.5 Исправление 20
3.2.6 Результат после исправления 20
3.2.7 Почему этот опыт важен для всей книги 20
3.3 Промежуточный итог после exp4 и exp5 21

4 Серия 6x: от topology correction к гибриду local + global **23**
4.1 Зачем вообще понадобилась серия 6x 23
4.2 exp6: topology-aware retrieval correction 23
4.2.1 Цель опыта 23
4.2.2 Часть A: хирургия топологии 23
4.2.3 Часть B: пост-аккумуляционные поправки 24
4.2.4 Результат 24
4.2.5 Что эксперимент реально доказал 24
4.3 exp6b: richer golden world и первое сравнение семейств 25
4.3.1 Почему одного кофейного fixture уже было мало 25
4.3.2 Четыре кластера графа 25
4.3.3 Формулы 25
4.3.4 Что можно было ожидать 25
4.3.5 Результат 26
4.3.6 Интерпретация 26
4.4 exp6c: mechanism benchmark для знаменателей 26
4.4.1 Зачем понадобился еще один шаг 26
4.4.2 Диагностические мотивы 26
4.4.3 Результат 27
4.4.4 Что добавил exp6c относительно exp6b 27

4.5 exp6d: глобальные priors поверх локального удержания 27
4.5.1 Почему после 6c вопрос еще не был закрыт 27
4.5.2 Методы 27
4.5.3 Ключевой мотив: ложная глобальная слава 28
4.5.4 Результат 28
4.5.5 Что особенно важно в этом результате 28
4.6 Что серия 6x дала и чего не дала 29
4.6.1 Что она действительно установила 29
4.6.2 Чего серия еще не установила 29

5 Серия 7x: перенос на эмбрион и локализация нового bottleneck 31
5.1 Почему перенос не был формальностью 31
5.2 exp7a: end-to-end transfer check 31
5.2.1 Замысел 31
5.2.2 Стенд 31
5.2.3 Что получилось вместо чистого transfer-benchmark 31
5.2.4 Результат 32
5.2.5 Почему это не был “нулевой” опыт 32
5.3 exp7b: controlled-source transfer 32
5.3.1 Почему это новый эксперимент, а не “починка 7a” 32
5.3.2 Два варианта графа 32
5.3.3 7b.1: первая controlled-source версия 33
5.3.4 7b.2: diagnostic-cue version 33
5.3.5 Результат 7b.2 33
5.3.6 Что именно теперь стало ясно 33
5.4 exp7c: conjunctive cue-overlap 34
5.4.1 Новый вопрос 34
5.4.2 Что проверялось 34
5.4.3 Результат 35
5.4.4 Почему это содержательно важно 35
5.5 Что серия 7x установила 35
5.5.1 Положительные результаты 35
5.5.2 Отрицательные результаты 35
5.5.3 Главный диагностический вывод 35

6 Серия 8a–9a: транспортная стена, similarity и достройка путей 37
6.1 Почему после 7x уже нельзя было просто искать еще один denominator 37
6.2 exp8a: bypass и repair как две попытки пробить transport wall 37

6.2.1 Замысел . . . 37
6.2.2 Методы . . . 37
6.2.3 Почему сравнение было честным . . . 38
6.2.4 Что именно проверялось . . . 38
6.2.5 Главный результат . . . 38
6.2.6 Что из этого следует . . . 39
6.2.7 Качественное чтение отдельных запросов . . . 39
6.3 exp8b: de-coning similarity как проектная развилка . . . 40
6.3.1 Статус . . . 40
6.3.2 Почему эта ветка возникла . . . 40
6.3.3 Что именно предполагалось проверить . . . 40
6.3.4 Почему даже неисполненная ветка все равно важна . . . 40
6.4 exp9a: dual-geometry transport и semantic-proximity edges . . . 41
6.4.1 Большой вопрос . . . 41
6.4.2 Постановка . . . 41
6.4.3 Главная сводка . . . 41
6.4.4 Как это читать . . . 42
6.4.5 Сильная сторона 9a . . . 42
6.4.6 Почему этого оказалось недостаточно . . . 42
6.4.7 Danger checks . . . 42
6.5 Что серия 8a–9a установила . . . 43
6.5.1 Отрицательные результаты . . . 43
6.5.2 Положительные и диагностические результаты . . . 43
6.5.3 Почти окончательный вывод этой линии . . . 43

7 Синтез: что доказала серия exp2–exp9a и куда она вывела 45
7.1 Четыре слоя выводов . . . 45
7.1.1 Слой 1: инфраструктура и эмбрион . . . 45
7.1.2 Слой 2: локальная механика retrieval . . . 45
7.1.3 Слой 3: глобальная структурная слава . . . 45
7.1.4 Слой 4: перенос на реальный эмбрион . . . 46
7.2 Что именно можно считать уже опровергнутым . . . 46
7.3 Что можно считать реально поддержанным . . . 46
7.4 Где сейчас настоящий bottleneck . . . 47
7.4.1 (1) Качество графового субстрата . . . 47
7.4.2 (2) Качество source injection . . . 47
7.4.3 (3) Предел самого spreading-подхода . . . 47
7.5 Почему серия все же продвинулась далеко . . . 48

7.6 Что логично делать дальше 48
7.6.1 Программа A: мета-аудит и смена класса retrieval 48
7.6.2 Программа B: архитектура консолидации эмбриона 48
7.6.3 Программа C: развести слои памяти 49
7.7 Финальная оценка серии 49

Предисловие

Этот текст нужен не для того, чтобы еще раз кратко пересказать ноутбуки, а для того, чтобы собрать из них единую исследовательскую историю. В серии `exp2`–`exp9a` происходило не линейное “улучшение метрики”, а медленное смещение вопроса. Сначала задача выглядела как инженерная: построить эмбрион памяти из реальных источников и проверить, что по нему вообще можно пустить сигнал. Потом стало ясно, что сам факт работоспособности пайплайна почти ничего не говорит о качестве retrieval. Затем выяснилось, что недостаточно просто добавить семантический gate: на уровне топологии и структуры сходимости граф ведет себя иначе, чем хотелось бы. После этого линия распалась на несколько ветвей:

1. ветвь локальной физики сигнала: что удерживает входящий поток и чем отличается узел-ответ от узла-хаба;
2. ветвь глобальных поправок: как отделить яркость, вызванную конкретным запросом, от яркости, вызванной общей структурной славой узла;
3. ветвь переноса на реальный эмбрион: сохраняются ли выводы, полученные на читабельных золотых мирах;
4. ветвь транспортной реконструкции: не в скоринге ли проблема, а в том, что сигнал вообще идет не туда или не доходит до цели;
5. ветвь качества самой геометрии сходства: не слишком ли грязен косинус в узком конусе NumberBatch-пространства;
6. ветвь реконструкции недостающих путей: можно ли достраивать граф семантическим каналом, не превратив его в обычный векторный поиск.

Во всех этих ветвях важно различать три уровня утверждений.

1. **Положительный результат.** Эксперимент убедительно показывает, что некоторый механизм действительно выполняет свою локальную работу.
2. **Отрицательный результат.** Эксперимент убедительно показывает, что некоторая популярная надежда не срабатывает.
3. **Диагностический результат.** Эксперимент не дает победителя, но сильно сужает пространство следующих гипотез.

Серия `exp2`–`exp9a` особенно богата именно третьим типом результата. Это не недостаток. Напротив, в архитектурной работе диагностический прогресс часто важнее преждевременной победы: он позволяет отличить настоящий предел механизма от случайного неуспеха конкретной настройки.

Вывод

Главная задача этого свода – не объявить задним числом, что серия “доказала”, а аккуратно восстановить логику движения. В ряде мест более ценно не то, что некоторый метод оказался лучшим, а то, что удалось честно показать: текущий тип механизма уперся в свой естественный потолок, и дальнейшее подкручивание тех же ручек уже не обещает качественного скачка.

Что именно считается экспериментом в этой книге

Здесь под “экспериментом” понимается не только исполненный ноутбук. В конце серии появляются и такие ветви, которые на момент свода существуют в разном статусе:

- полностью исполненный ноутбук с сохраненными output-ячейками и отчетом;
- исполненный ноутбук, но без окончательного markdown-репорта;
- проектная развилка, подробно спроектированная как следующий опыт, но еще не прогнанная.

Это особенно важно для `exp8b`. Его нельзя молча подать как уже завершенный опыт. Но и выбрасывать его из исследовательской линии было бы ошибкой: он играет роль концептуального фильтра, который должен был отделить дефекты транспортной архитектуры от дефектов самой embedding-геометрии.

Как читать этот текст

Есть два разумных маршрута чтения.

1. **Линейный маршрут.** Читать подряд, как историю исследования. Тогда лучше начинать с введения и не перепрыгивать через главы, потому что поздние серии (`7x`–`9a`) содержательно опираются на различия, выработанные в `6x`.
2. **Маршрут по вопросам.** Если интересует одна конкретная тема – например, PageRank-поправка, capacity, overlap cue-fields или semantic proximity edges, – можно идти сразу в соответствующую серию. Но в этом случае нужно помнить, что поздние главы используют терминологию, введенную в первых двух главах.

Стилистическое замечание

Язык текста намеренно педагогичен, но не упрощен до уровня популяризации. Читатель здесь предполагается математически и алгоритмически подготовленным. Поэтому базовые понятия вроде графа, случайного блуждания, косинусной схожести или стационарного распределения не разжевываются с нуля; вместо этого основной акцент сделан на том, как именно эти объекты начинают

работать внутри архитектуры памяти и почему в одном месте они помогают, а в другом создают иллюзию прогресса.

1. Исследовательская программа: граф, сигнал, метрики

1.1. Почему серия вообще начинается с эмбриона

В архитектуре RIN графовая память не возникает как чистая абстракция. Она должна рано или поздно стать рабочим субстратом, по которому проходит сигнал, куда коммитятся новые следы и из которого извлекается контекст. Поэтому уже начиная со второго эксперимента задача формулируется двойственно:

1. нужен **эмбриональный граф**, достаточно реальный, чтобы на нем вылезали содержательные дефекты;
2. нужен **контролируемый стенд**, достаточно малый и читаемый, чтобы на нем можно было измерять не только факт прохождения сигнала, но и качество retrieval.

Отсюда вся дальнейшая архитектура серии:

- реальные источники – для столкновения с настоящей структурной грязью;
- золотые синтетические миры – для локализации конкретных механизмов;
- перенос с золотого мира на эмбрион – как проверка того, переживает ли формула выход из лабораторной колбы.

1.2. Как выглядит базовый графовый объект

Во всех сериях используется одна и та же базовая идея памяти как направленного взвешенного графа. Узлы представляют концепты, события, состояния, а позднее – эпизодические или служебные элементы. Ребра несут тип и вес.

Минимально полезные поля узла:

- текстовое содержимое;
- тип узла (`concept`, `episode`, `claim`, `relationship_state` и т. д.);
- embedding в общем векторном пространстве;
- метаданные происхождения и эпистемического статуса.

Минимально полезные поля ребра:

- источник и приемник;

- тип отношения (`IsA`, `RelatedTo`, `UsedFor`, `Causes`, `HasProperty` и пр.);
- вес проводимости.

Уже на этом уровне скрыта важная методологическая ловушка. Один и тот же граф можно рассматривать по меньшей мере в трех разных ролях:

1. как **хранилище фактов и эпизодов**;
2. как **среду распространения сигнала**;
3. как **объект переписывания и консолидации**.

Серия `exp2`–`exp9a` почти целиком посвящена второй роли, но время от времени касается первой и третьей. Из-за этого полезно заранее различать операторы чтения и записи.

1.3. Как выглядит retrieval в базовой версии

Пусть задан набор стартовых якорей. Им присваивается начальная амплитуда, после чего по графу запускается затухающее распространение. В простейшей форме можно думать о шаге диффузии как о правиле

$$a_{t+1}(v) = \lambda \sum_{u \to v} a_t(u)\, P(u \to v),$$

где:

- $a_t(v)$ – уровень активации узла v на шаге t;
- λ – коэффициент затухания;
- $P(u \to v)$ – нормированная доля потока, идущая из u в v.

В реальных ноутбуках используется не буквальная марковская цепь с бесконечным пределом, а конечный процесс с отсечением слабых хвостов. Практически это значит следующее:

1. в стартовые узлы впрыскивается фиксированный заряд;
2. на каждом шаге активны только узлы выше минимального порога;
3. выходящий поток делится между исходящими ребрами пропорционально их весу;
4. процесс идет ограниченное число шагов или до опустошения фронта.

Из этого уже видно, почему серия так быстро вышла к проблеме хабов. Узел может оказаться ярким по двум разным причинам:

1. он действительно хорошо объясняется данным запросом;
2. в него удобно сходится много путей, даже если по смыслу он слишком общий.

1.4. Три уровня retrieval-скоринга

По ходу серии постепенно кристаллизуется удобное разложение retrieval на три содержательно разных уровня.

1.4.1. Локальный транспорт

Это вопрос о том, *дошел ли сигнал вообще* и как он растекся по имеющимся путям. Здесь живут:

- исходные якоря;
- затухание;
- веса ребер;
- типизация проводимости;
- возможные дополнительные каналы вроде semantic edges.

1.4.2. Локальное удержание

Даже если два узла получили одинаковый приток, не обязано быть так, что они одинаково *удерживают* этот приток как локально значимую активацию. Отсюда рождаются знаменатели семейства $R(B)$:

$$\text{retained}(B) = \frac{\text{incoming}(B)}{R(B)}.$$

На этом уровне обсуждаются:

- степень узла;
- взвешенная исходящая сила;
- активный fan-in;
- локальная capacity как “эффективный объем представления”.

1.4.3. Глобальная структурная слава

Наконец, даже хороший локальный отклик может оказаться слишком высоким просто потому, что узел глобально знаменит. Тогда появляется еще одна поправка:

$$\text{Score}(v \mid q) = \frac{\text{LocalResponse}(v \mid q)}{\text{Prior}(v)^{\eta}}.$$

Именно здесь возникают in-strength, out-strength, PageRank и их сравнение.

Вывод

Одно из важнейших достижений серии состоит в том, что эти три уровня перестали смешиваться. В начале работ любой неуспех мог быть списан на “плохие эмбеддинги” или “неудачную формулу”. К середине серии стало ясно, что транспорт, удержание и глобальная слава – разные механизмы, и провал любого из них не лечится простой правкой соседнего.

1.5. Роль embedding-пространства

Embedding в этой серии играет несколько ролей сразу.

1. Он нужен для anchoring: чтобы query-слова можно было связать с узлами графа.
2. Он нужен для семантического gate: чтобы весить релевантность узла текущему запросу.
3. Он нужен для additive bypass в `8a`: similarity начинает напрямую входить в итоговый score.
4. Он нужен для semantic-edge construction в `9a`: ближайшие соседи в embedding-пространстве становятся кандидатами на дополнительные ребра.

Это делает особенно опасной ситуацию, когда embedding-пространство само по себе дефектно. Если косинус живет в узком конусе, generic nodes могут становиться псевдохабами еще до того, как сигнал вообще вошел в графовую динамику. Поэтому ветка `8b` возникает не как побочная эстетика, а как попытка убрать важный конфаундер: не следует ли приписываемый графу дефект на самом деле свойствам самой similarity-геометрии.

1.6. Какими метриками пользовалась серия

Ранняя часть серии опиралась на простые и очень понятные критерии:

- правильный узел должен быть выше отвлекающего;
- broad hub не должен побеждать специфический ответ, если запрос не о нем;
- наоборот, legitimate hub не должен быть задавлен там, где запрос именно о нем.

Позднее появляются более тонкие метрики:

- число пройденных pairwise inequality;
- число решенных запросов;
- margin между лучшим правильным и лучшим distractor;
- mean rank и MRR;

- manifold-aware success вместо проверки одного канонического узла;
- специальные danger-checks: hub reflood и RAG collapse.

Сюда стоит добавить два методологических принципа, выработанных по ходу серии.

Методический акцент

Принцип 1. Нельзя ранжировать методы по “суммарному margin”, если они по-разному масштабируют одну и ту же базовую картину. Иначе получится ложная победа просто более агрессивной нормировки.
Принцип 2. Как только граф начинает выдавать не один канонический узел, а соседний семантический manifold, оценка должна это учитывать. Иначе мы будем объявлять провалом ситуацию, в которой память по существу попала в правильный район, но не в ту формулировку вершины, которую человек заранее выписал в таблицу ответов.

1.7. Почему понадобились золотые миры

Реальный граф нужен для реализма, но он почти бесполезен как чистый измеритель retrieval-качества, если там нет ground truth. Поэтому серия несколько раз возвращается к hand-authored worlds. Это не “игра в песочнице” в плохом смысле, а нормальный исследовательский ход:

1. на реальном графе вылавливаются инфраструктурные и содержательные дефекты;
2. на золотом мире локализуется механизм;
3. затем гипотеза переносится обратно на более реальный срез.

Слабость золотого мира известна заранее: он зависит от авторского конструирования. Его сила тоже известна заранее: только он позволяет быстро различить, является ли неуспех следствием плохой формулы, плохого графа или грязной постановки.

1.8. Как устроен дальнейший рассказ

Оставшиеся главы идут не просто по номерам, а по логическим сериям.

- Главы exp2–exp5 строят базовые объекты: эмбриональный граф, золотой мир, семантический gate и write-path.
- Глава о 6x разбирает локальное удержание и глобальные priors.
- Глава о 7x показывает, как эти идеи переносятся на реальный эмбрион и где именно перенос начинает ломаться.
- Глава о 8a–9a фиксирует смещение от “оптимизации знаменателя” к проблеме транспортной архитектуры и реконструкции путей.
- Заключительная глава собирает общую карту того, что серия действительно доказала и где ее естественная развилка дальше.

2. От эмбриона к золотому миру: exp2 и exp3

2.1. exp2: эмбриональный граф из реальных источников

2.1.1. Смысл эксперимента

exp2 был первым настоящим столкновением с вопросом: можно ли вообще собрать из реальных источников такой граф, который будет не просто лежать на диске, а поддерживать полный Stage-1 pipeline. Речь шла не о красоте retrieval, а о минимальной жизнеспособности всей линии:

1. seeded loading подграфа из реальных данных;
2. anchoring пользовательского запроса в этом графе;
3. диффузия активации;
4. упаковка результата в ограниченный контекст.

Важное отличие от поздних экспериментов состоит в том, что здесь еще нет полноценной ground-truth оценки качества ответа. Эксперимент отвечает на более ранний вопрос: работает ли вообще “физика системы” на настоящем материале.

2.1.2. Стенд

Источники были следующими:

- полный дамп ConceptNet;
- агрегированный ATOMIC 2019;
- английский NumberBatch.

Загрузка строилась как seeded BFS от ограниченного набора входов с радиусом 3–4 и потолком числа concept-узлов. Дополнительно использовались:

- отдельный более жесткий порог для `RelatedTo`;
- точное лемматическое связывание ATOMIC с отключенным fuzzy fallback;
- общий векторный словарь NumberBatch для якорения и similarity.

В терминах будущей архитектуры этот опыт проверял, может ли эмбрион *существовать как вычислимый объект*, а не лишь как схема в manual.

2.1.3. Что здесь могло получиться

На старте было две альтернативы.

1. **Оптимистическая.** Пайплайн в целом работает, а проблемы будут касаться главным образом качества retrieval.
2. **Пессимистическая.** Реальные данные выявят системные дефекты еще до момента, когда вообще можно говорить о retrieval-качестве: ошибки разметки, схлопывание подписей, фантомные хабы, неработающие embedding lookup и т. п.

2.1.4. Что реально произошло

Произошел более сильный и более полезный вариант пессимистической развилки: pipeline в целом заработал, но именно на нем всплыл целый класс ошибок, которые синтетический мир не мог бы показать.

(1) Ошибка labeling из POS-tagged URI

Часть узлов получала содержимое вроде одной буквы `n`, потому что последний сегмент URI принимался за label узла, хотя это был POS-tag. Ошибка оказалась не косметической: в одном из запусков она затронула около трети узлов.

(2) Дублирование ребер

Двунаправленное индексирование adjacency делало одно и то же ребро доступным из обоих концов, а BFS потом повторно добавлял его в граф. Нужна была дедупликация по assertion id самого дампа.

(3) Производство хабов связью `RelatedTo`

Это одна из ключевых находок всей ранней серии. Оказалось, что проблема не в “больших степенях вообще”, а в том, что именно `RelatedTo` на реальных данных образует слишком широкие и семантически рыхлые маршруты. Отсюда уже в exp2 родилась мысль, что тип отношения в графе не является декоративной меткой: он меняет реальную физику retrieval.

(4) Несовпадение формата NumberBatch и ConceptNet-URI

English-only NumberBatch оказался записан как bare terms, а не как полные ConceptNet URI. Поэтому все lookup молча проваливались, и embeddings не подгружались вовсе.

(5) Переизбыточное ATOMIC-linking

Даже после исправления NumberBatch слишком мягкая схема embedding-based linking присоединяла к графу лавину эпизодов. Переход к exact lemma match дал резкое сжатие числа episode nodes и тем самым показал, что на этом

уровне важно не “чуть лучше подобрать threshold”, а сначала вообще вырваться из режима неконтролируемого фан-аута.

2.1.5. Инфраструктурный результат

После исправлений pipeline заработал end-to-end на реальных данных. Запрос “what is the difference between lisbon and bacalhau” успешно якорился, распространял активацию и упаковывался в контекст.

Но этот результат сам по себе не означал, что retrieval хорош. Он означал более скромную и при этом чрезвычайно важную вещь: архитектура вышла из стадии чистой спекуляции и впервые столкнулась с реальной семантической грязью.

Вывод

exp2 доказал не качество памяти, а жизнеспособность эмбриона как вычислимого объекта. Еще важнее он доказал, что реальные данные порождают собственный класс багов, которых не существует на игрушечных графах. Это и обосновывает всю дальнейшую линию: без реального эмбриона часть архитектурных дефектов просто оставалась бы невидимой.

2.1.6. Что эксперимент оставил открытым

Главный открытый вопрос после exp2: что значит “хороший” retrieval на реальном графе, если там нет ground truth. Сомнительный ranking можно заметить, но нельзя строго назвать неверным без контролируемой постановки.

Именно отсюда рождается exp3. Нужен был не новый retrieval law, а маленький золотой мир, где правильный ответ заранее известен.

2.2. exp3: золотой микромир как измеритель retrieval

2.2.1. Зачем понадобился второй стенд

После exp2 стало понятно, что два вопроса нельзя больше смешивать:

1. может ли pipeline работать на реальных данных;
2. выдает ли он правильный ответ, когда правильность вообще можно проверить.

exp3 не вводит новый retrieval-механизм. Он вводит новую *измерительную среду*: маленький hand-authored граф, где каждая неудача может быть названа по имени.

2.2.2. Конструкция золотого мира

Граф был намеренно очень маленьким и семантически прозрачным. В нем были два сюжетных кластера.

- Кофейный: `coffee`, `caffeine`, `beverage`, `hot drink`, `tea`, `juice`, `soda`.

- Географический: north dakota, south dakota, north.

Смысл кластера про кофе был особенно важен. Он позволял сравнить:

- специфический, содержательно правильный ответ caffeine;
- широкий, структурно связный, но концептуально грубый distractor beverage.

2.2.3. Какие именно сценарии проверялись

В exp3 было четыре сценария.

1. **Multi-word anchoring.** Проверка, что north dakota якорится как единый термин, а не распадается на north и dakota.
2. **Distractor vs answer.** Ключевой сценарий: “why coffee gives energy” не произносит caffeine явно, поэтому retrieval обязан выйти к нему за счет графовой структуры и similarity, а не за счет банального подглядывания.
3. **Multi-anchor query.** Проверка, что разные темы могут жить вместе в одном контексте.
4. **Regression on irrelevant words.** Проверка, что stopwords и служебные слова не ломают anchoring.

2.2.4. Главный результат

Сценарии 1, 3 и 4 прошли хорошо. Но второй сценарий дал первый по-настоящему диагностический удар по retrieval-механике.

Для запроса про кофе и энергию peak activation выглядела так:

Узел	Амплитуда	Роль
beverage	146.67	distractor
coffee	100.00	anchor
hot drink	100.00	anchor
tea	100.00	anchor
caffeine	26.67	правильный ответ

Это не “слегка неудачный ranking”, а качественно неправильное поведение: широкий хаб побеждает специфический ответ более чем впятеро.

2.2.5. Что именно это доказало

Важен не только сам факт ошибки, но и ее структура. caffeine проиграл не потому, что он был плохо представлен как embedding. Он проиграл потому, что в базовой диффузии нет механизма, который бы делал различие между:

- узлом, в который сходится много путей просто по топологии;
- узлом, который является более точным ответом на вопрос.

Это сразу породило рабочую гипотезу для exp4: возможно, хватает просто добавить query-conditioned semantic gate, то есть умножать поток на релевантность узла запросу.

Риск интерпретации

Именно здесь серия могла бы свернуть в ложную сторону и начать считать, что проблема целиком в embeddings. Но exp3 устроен так, что дефект уже виден на очень маленьком и прозрачном графе. Поэтому рабочая гипотеза после него – не “надо срочно менять векторное пространство”, а “в operator diffusion отсутствует релевантностный множитель”.

2.2.6. Связь exp2 и exp3

Пара exp2–exp3 образует очень важное методологическое кольцо.

- exp2 показал, что без реального контента не увидеть целый класс инфраструктурных дефектов.
- exp3 показал, что без золотого мира нельзя всерьез говорить о качестве retrieval, даже если pipeline уже технически работает.

Вместе они вводят основной стиль всей дальнейшей серии: чередование между реализмом и контролируемостью.

3. От семантического gate к write-path: exp4 и exp5

3.1. exp4: semantic gate как первая попытка исправить retrieval

3.1.1. Что именно проверялось

После exp3 гипотеза выглядела почти естественно: если диффузия никак не учитывает релевантность узла запросу, то, может быть, достаточно встроить в нее множитель

$$\mathrm{rel}(B \mid q),$$

зависящий от similarity между узлом B и query-векторами.

Тогда переходный вклад вдоль ребра будет определяться не только весом ребра, но и тем, насколько приемник вообще подходит к смыслу запроса.

3.1.2. Постановка

Стенд сохранялся максимально близким к exp3:

- тот же hand-authored кофейный граф;
- тот же запрос “why coffee gives energy”;
- те же embeddings;
- единственное изменение – наличие или отсутствие semantic gate.

Это был хороший чистый опыт: если ranking исправится, значит, проблема действительно была только в том, что диффузия игнорировала query relevance.

3.1.3. Что ожидалось при двух конкурирующих картинах

1. **Картина A: причина в отсутствии relevance-gate.** Тогда после добавления gate caffeine должен обогнать beverage.
2. **Картина B: причина не только в relevance-gate, но и в самой топологии сходимости путей.** Тогда gate улучшит картину, но не перевернет ее.

3.1.4. Результат

Вторая картина оказалась верной. После введения gate распределение изменилось в нужную сторону, но не настолько, чтобы исправить ranking:

Узел	Без gate	С gate
beverage	146.67	129.46
caffeine	26.67	27.20
juice	23.47	11.49
soda	23.47	13.65

То есть gate работал *качественно правильно*, но *количественно недостаточно*.

3.1.5. Почему это особенно важный результат

Здесь произошла одна из самых тонких и самых полезных развязок всей ранней серии. Можно было бы поспешно сказать:

> “Ну вот, similarity плохая. Посмотрим на слово energy и увидим, что beverage к нему даже ближе, чем caffeine. Значит, виновато embedding-пространство”.

Но аккуратный разбор показал, что это ложный след. По фактическому правилу агрегации $\mathrm{rel}(B \mid q)$ узел caffeine *и так* был релевантнее beverage. Проигрыш происходил не на уровне per-node relevance, а на уровне *суммарной амплитуды после сходимости нескольких путей*.

Вывод

exp4 показал очень важную вещь: можно иметь узел, который *сам по себе* более релевантен запросу, и все равно проигрывать, если конкурирующий узел получает поток по нескольким сходящимся маршрутам. Это первый явный выход серии к проблеме convergence topology.

3.1.6. Какой следующий вопрос возник немедленно

После exp4 стало недостаточно обсуждать только similarity. Требовался эксперимент, который бы изолировал следующее утверждение:

> “Если менять только топологию сходимости, оставляя запрос, embeddings и gate неизменными, то ranking действительно должен переворачиваться”.

Это и становится задачей exp6; но до него в серии появляется еще одна ветвь – exp5, посвященная не retrieval, а write-path.

3.2. exp5: write-path как отдельный архитектурный срез

3.2.1. Почему эта глава здесь вообще стоит

На первый взгляд exp5 выбивается из серии. Вокруг него нет ни caffeine, ни PageRank, ни topology correction. Но именно это и важно: память не может состоять только из retrieval. Нужен механизм записи новых узлов, дедупликации, коммита и форензического поиска по времени.

exp5 поэтому играет роль боковой, но принципиально важной ветви: он проверяет, что граф не только можно читать, но и можно *корректно модифицировать*.

3.2.2. Постановка

Сценарий был полностью контролируемым. Вместо автоматического извлечения кандидатов из естественного текста вручную задавались объекты `Candidate`, имитирующие результат будущего экстрактора.

Сюжет – “поездка на Азоры”:

1. покупка плавок для поездки;
2. повтор того же события, который должен быть опознан как дубликат;
3. покупка зонта для той же поездки;
4. отдельное событие про ноутбук в другой день.

Проверялись:

- корректный коммит узлов;
- пропуск эпистемического дубликата;
- forensic index по timestamp;
- ситуация, где один кандидат ссылается на другой кандидат из того же batch.

3.2.3. Какая альтернатива тут была на самом деле

Хотя сценарий выглядит техническим, в нем скрыта содержательная архитектурная развилка.

1. **Наивный single-pass commit.** Кандидаты коммитятся и сразу разрешают свои ребра.
2. **Batch-aware two-pass commit.** Сначала весь пакет кандидатов проходит gate и получает id, а уже потом разрешаются их внутренние ссылки.

Если второй режим не нужен, значит внутренняя структура event-batch не играет роли. Если нужен, значит write-path обязан уважать пакетность события.

3.2.4. Что обнаружилось

На первом же прогоне всплыл настоящий баг: если первый кандидат ссылается на сущность, которую второй кандидат того же batch должен определить как полноценный узел, single-pass логика создает stub и тем самым портит последующий commit.

В конкретном сценарии это означало, что `azores trip` оказывался не `episode`-узлом, а generic `claim`-stub. Более того, правильный кандидат затем ошибочно считался дубликатом уже существующего узла.

3.2.5. Исправление

Исправление состояло в переходе к двухпроходной логике:

1. весь batch проходит gating;
2. все одобренные кандидаты создают голые узлы;
3. внутри batch строится `content -> id` map;
4. только после этого разрешаются ребра, причем сначала по этой карте, и лишь затем через механизм stub.

3.2.6. Результат после исправления

После введения двухпроходного коммита сценарий прошел чисто:

- оба нужных узла коммитились корректно;
- дубликат пропускался;
- `azores trip` получал правильный `node_kind`;
- forensic lookup по дню 9 находил только релевантные узлы и не подмешивал ноутбук из другого дня.

3.2.7. Почему этот опыт важен для всей книги

exp5 показывает, что у архитектуры памяти есть как минимум две разные линии зрелости:

- retrieval-линия, где исследуется прохождение и ранжирование сигнала;
- write-path-линия, где исследуется корректность коммита и структуры обновления графа.

Именно благодаря этому exp5 важно не потерять в общем своде. Без него может возникнуть ложное впечатление, что серия занималась только чтением памяти. На деле уже здесь проясняется, что даже простой commit требует не меньшей аккуратности, чем retrieval.

Открытый вопрос

exp5 оставил открытым следующий вопрос: что будет, когда ссылки начнут идти не только внутри одного batch, но и между несколькими событиями, особенно если ранний event сначала породил stub, а поздний должен его доопределить. Это уже не проблема локального коммита, а проблема долговременной консолидации и ревизии графа.

3.3. Промежуточный итог после exp4 и exp5

К этому моменту исследовательская линия уже расщепилась на две зрелые оси.

1. Retrieval-ось показала, что простой semantic gate не решает проблему слишком ярких широких узлов.
2. Write-path-ось показала, что даже controlled commit требует batch-aware-процедуры и не выдерживает наивного single-pass режима.

Дальнейшая серия 6x возвращается именно к retrieval-оси и начинает разбирать ее уже более математически: что именно удерживает сигнал, и почему некоторые узлы светятся слишком ярко не по смыслу, а по топологии и глобальной славе.

4. Серия 6x: от topology correction к гибриду local + global

4.1. Зачем вообще понадобилась серия 6x

После exp4 стало ясно, что семантический gate сам по себе не вытягивает специфический ответ из-под широкого хаба. Но это еще не говорило, *какой именно* механизм отсутствует. Серия 6x строится как последовательное расщепление вопроса:

1. действительно ли дело в топологии сходимости путей;
2. если да, можно ли исправить это локальной поправкой на удержание;
3. если локальная поправка помогает, какой именно класс знаменателей осмыслен;
4. если локального удержания мало, нужен ли еще и глобальный structural prior.

Важно, что здесь впервые появляется уже почти “физическая” интерпретация узла: не всякий inflow одинаково превращается в salience, потому что разные узлы по-разному удерживают и рассеивают приходящий поток.

4.2. exp6: topology-aware retrieval correction

4.2.1. Цель опыта

exp6 должен был сделать то, что exp4 еще не мог: отделить собственно semantic relevance от эффекта многократной сходимости на хабе.

Для этого опыт делился на две части:

1. topology surgery: меняется только структура путей;
2. correction ablation: исходный граф сохраняется, меняется только формула поправки.

4.2.2. Часть A: хирургия топологии

В первой части проверялась почти причинная гипотеза:

> если проблема действительно в количестве и силе сходящихся маршрутов, то достаточно удалить пару входов в `beverage` или добавить пару входов в `caffeine`, и ranking должен перевернуться даже без смены embeddings и query.

Именно это и произошло.

Вариант графа	`caffeine`	`beverage`	Победа
Исходный	27.20	129.46	нет
Срезать часть входов в `beverage`	37.57	27.38	да
Добавить входы в `caffeine`	95.10	77.17	да
Сделать оба изменения	141.30	19.26	да

Это один из самых чистых результатов всей серии: при неизменном query и неизменном semantic gate ranking переворачивается одной лишь правкой topology.

4.2.3. Часть B: пост-аккумуляционные поправки

После этого можно было уже предметно тестировать корректирующие формулы. На исходном графе сравнивались:

- отсутствие поправки;
- деление на $\deg(B)^{\gamma}$;
- деление на $\text{fanin}_t(B)^{\gamma}$.

Ключевым было то, что штраф накладывался *после слияния вкладов* в узле назначения. Это принципиально: если наказывать поток локально у источника, то узел, у которого есть единственный путь наружу, все равно отправит весь свой заряд в этот путь.

4.2.4. Результат

Единственной формулой, действительно перевернувшей ranking, оказалось деление на полную степень узла:

$$\text{incoming}(B) \mapsto \frac{\text{incoming}(B)}{\deg(B)}.$$

Это был грубый, но убедительный proof of concept.

4.2.5. Что эксперимент реально доказал

`exp6` не доказал, что $1/\deg(B)$ есть закон памяти. Он доказал более узкое, но очень важное утверждение:

> если узел побеждает за счет чрезмерной сходимости путей, то поправка должна действовать на merged accumulation, а не только на локальный выбор ребра у источника.

4.3. exp6b: richer golden world и первое сравнение семейств

4.3.1. Почему одного кофейного fixture уже было мало

Успех на одной конфигурации caffeine против beverage еще не говорит, что найдена хорошая retrieval law. Можно легко подобрать формулу, которая работает только на одном сценарии.

Поэтому exp6b строит богатый, но все еще читаемый золотой мир на 33 узлах. Его смысл не в “реализме мира”, а в разнообразии структурных ситуаций.

4.3.2. Четыре кластера графа

В графе были выделены:

- домашний кластер (apartment, wallet, keys, cats, feeding alarm, elevator turnaround);
- travel cluster (trip, airport, passport, boarding pass, hotel);
- work cluster (office, meeting, laptop, charger, dead laptop);
- coffee/health cluster (coffee, caffeine, beverage, energy, water).

Принципиально важно, что запросы включали не только anti-hub cases, но и legitimate-hub cases. Иначе любую агрессивную нормировку можно было бы ошибочно объявить хорошей.

4.3.3. Формулы

Появляются семейства

$$R_0(B) = 1,$$

$$R_1(B) = 1 + \beta\, k_{\text{out}}(B)^{\gamma},$$

$$R_2(B) = 1 + \beta\, s_{\text{out}}(B)^{\gamma},$$

$$R_3(B) = \alpha + \beta\, s_{\text{out}}(B)^{\gamma},$$

$$R_4(B) = C(B) + \beta\, s_{\text{out}}(B)^{\gamma},$$

$$R_5(B) = \alpha + C(B) + \beta\, s_{\text{out}}(B)^{\gamma}.$$

Здесь впервые появляется capacity $C(B)$ как локальная ширина представления.

4.3.4. Что можно было ожидать

Если “утечка” исчерпывает всю проблему, должен был победить leak-only класс R_2 или R_3. Если же разные узлы отличаются не только по суммарной силе исходящих каналов, но и по собственной концептуальной ширине, тогда capacity должна дать добавочное преимущество.

4.3.5. Результат

В exp6b лучшим in-sample и leave-one-out победителем стало семейство

$$R_4(B) = C(B) + 0.1\, s_{\text{out}}(B)^{0.5}.$$

Оно решило все 7 запросов и дало наибольший общий margin. Базовый raw case проваливал только 6/7, а leak-only формулы не смогли стать убедительными победителями.

4.3.6. Интерпретация

Это был первый сильный аргумент в пользу следующего тезиса:

> узел может быть широким в двух разных смыслах: потому что у него много исходящих каналов и потому что он сам по себе представляет более объемное понятие.

Именно эта двойственность и пыталась поймать пара “capacity + leak”.

Риск интерпретации

exp6b остается синтетическим опытом. Capacity в нем ручная. Поэтому после него нельзя было честно говорить “мы нашли закон памяти”. Можно было говорить только “семейство без capacity уже не выглядит достаточным”.

4.4. exp6c: mechanism benchmark для знаменателей

4.4.1. Зачем понадобился еще один шаг

У exp6b оставалось методологическое возражение: возможно, ответ приходил не столько из-за хорошего знаменателя, сколько из-за удачной geometry anchors и semantic gate. Поэтому exp6c радикально очищает постановку:

- для каждого запроса заранее задан source event;
- после этого диффузия идет без query-conditioned gating;
- benchmark измеряет именно поведение знаменателя.

4.4.2. Диагностические мотивы

Здесь уже не большой смешанный мир, а набор маленьких мотивов, каждый из которых стрессует отдельное различие:

- weather motif – проверка weighted leakage;
- kit motif – наказание слишком грубого degree counting;
- travel motif – чистый capacity-only case;
- health motif – перенос исходного caffeine/beverage конфликта в новый стенд.

4.4.3. Результат

В exp6c baseline и degree-control выступили слабо:

- R_0 решил 2/5 задач;
- R_1 в leave-one-out вообще рухнул до 1/5;
- weighted leak alone (R_2, R_3) улучшил ситуацию, но не дотянул до полной стабильности;
- capacity-aware families (R_4, R_5) оказались единственными по-настоящему сильными кандидатами.

Лучшим leave-one-out вариантом стал

$$R_5(B) = 1.0 + C(B) + 0.05\, s_{\text{out}}(B)^{0.5}.$$

4.4.4. Что добавил exp6c относительно exp6b

exp6b говорил: “capacity выглядит полезной”. exp6c добавил: “capacity делает свою работу даже тогда, когда anchor geometry убрана из игры”.

Это уже не просто победа в смешанном benchmark, а настоящий mechanism audit.

4.5. exp6d: глобальные priors поверх локального удержания

4.5.1. Почему после 6c вопрос еще не был закрыт

Даже если узел хорошо удерживает локальный приток, он может быть ярким просто потому, что глобально структурно знаменит. Поэтому возникает следующий шаг:

$$\text{Score}(v \mid q) = \frac{\text{LocalResponse}(v \mid q)}{\text{Prior}(v)^{\eta}}.$$

Содержательный вопрос уже не в том, “заменяет ли PageRank локальную физику”, а в том, добавляет ли глобальный prior *нечто не сводимое* к локальному удержанию.

4.5.2. Методы

Сравнивались:

- $P0$: raw diffusion;
- $P1$: raw / out-strength$^{\eta}$;
- $P2$: raw / in-strength$^{\eta}$;
- $P3$: raw / PageRank$^{\eta}$;

- $P4$: local-only retained diffusion;
- $P5$: local retained diffusion / PageRank$^{\eta}$.

При этом локальная часть в $P4$ и $P5$ фиксировалась на победителе из 6c.

4.5.3. Ключевой мотив: ложная глобальная слава

Особенно важным был morning motif, где water делали глобально известным за счет множества дальних входящих связей, но локальный утренний cue делали почти симметричным между water и espresso. Этот мотив проверял, может ли prior отличить:

- “узел яркий, потому что глобально везде встречается”;
- “узел яркий, потому что именно здесь он нужен”.

4.5.4. Результат

Лучшим методом стал гибрид

$$P5(v \mid q) = \frac{\text{LocalResponse}(v \mid q)}{\text{PR}(v)^{0.25}},$$

где LocalResponse задается capacity-aware local denominator из 6c.

Этот гибрид решил все 7 задач и in-sample, и leave-one-out.

4.5.5. Что особенно важно в этом результате

Успех P5 означает не “PageRank победил”, а нечто более тонкое:

1. purely global penalties полезны;
2. purely local retention полезна;
3. но наилучшее поведение дает их комбинация;
4. при этом η должен быть мягким: полная PMI-подобная сила оказывается слишком агрессивной.

Вывод

К концу серии 6x складывается первая действительно зрелая retrieval картина:

$$\text{Score}(v \mid q) \approx \frac{\text{incoming}(v \mid q)}{R_{\text{local}}(v)\,\text{PR}(v)^{\eta}}.$$

Локальная часть отвечает за удержание и противодействие smearing, глобальная – за снятие structural fame. Это еще не перенос на реальный граф, но это уже чистая и достаточно сильная рабочая гипотеза.

4.6. Что серия 6x дала и чего не дала

4.6.1. Что она действительно установила

1. Пост-аккумуляционная topology-aware correction имеет смысл.
2. Простая степень узла может работать как грубый control, но она слишком тупа, чтобы считаться итоговым законом.
3. Weighted leak полезен, но сам по себе не исчерпывает задачу.
4. Capacity выполняет не декоративную, а содержательную работу.
5. Глобальный prior, особенно мягкий PageRank, добавляет нечто реальное поверх локальной физики.

4.6.2. Чего серия еще не установила

1. Что manual capacity переносима на реальный граф.
2. Что найденная формула не является артефактом hand-authored motifs.
3. Что hybrid local + global сохранит преимущество на шумном эмбриональном подграфе.

Именно поэтому после 6d естественным следующим ходом становится не очередной synthetic benchmark, а transfer-check на реальном seeded subgraph.

5. Серия 7x: перенос на эмбрион и локализация нового bottleneck

5.1. Почему перенос не был формальностью

После 6d легко было бы вообразить, что осталось просто применить победивший гибрид к большому реальному графу. Но именно здесь и начинается самая поучительная часть всей серии. Перенос показал, что формула, убедительная на золотом мире, еще не гарантирует содержательного прогресса на эмбрионе. Причина при этом оказалась не тривиальной.

5.2. exp7a: end-to-end transfer check

5.2.1. Замысел

exp7a спрашивал: сохраняются ли выводы 6c–6d при переходе на крупный seeded subgraph, где нет ручной capacity и где retrieval должен работать на реальной, а не hand-authored топологии.

Сравнивались raw diffusion, in-strength prior, PageRank prior, local leak only и hybrid local leak + PageRank.

5.2.2. Стенд

Подграф строился как seeded expansion по понятиям, продолжающим мотивы 6x: кофе, паспорт, apartment, charger, travel, morning и т. д. Формально ceiling на concept nodes составлял 10 000, но включение ATOMIC создало гораздо более массивную оболочку episode nodes.

5.2.3. Что получилось вместо чистого transfer-benchmark

Фактический размер графа оказался таким:

- 10 000 concept nodes;
- 455 481 episode nodes;
- 465 481 узел суммарно.

Уже одно это сильно меняет характер задачи. Но еще важнее был другой эффект: automatic anchoring порождал шумные source clouds вроде thing, matter, before, early, imbibement, morningness.

5.2.4. Результат

Все tested methods дали одинаковую дискретную картину:

- 4/13 pairwise checks;
- 3/7 solved queries.

Численные margins различались сильно, но ordering не менялся.

5.2.5. Почему это не был “нулевой” опыт

На первый взгляд такой tie можно было бы счесть провалом. Но на деле exp7a дал один из важнейших диагнозов серии:

> в реальном end-to-end режиме retrieval law почти не тестируется, потому что pipeline уже ломается на этапе query → source injection.

Кроме того, top-k были засорены echo самих source nodes, так что наблюдаемое “retrieval” часто было просто возвратом к уже впрыснутым якорям.

Вывод

exp7a не опроверг local+global hybrid. Он показал, что на реальном эмбрионе более ранний слой – anchoring и source injection – доминирует над всем, что происходит позже. Это и стало основанием для controlled-source опыта 7b.

5.3. exp7b: controlled-source transfer

5.3.1. Почему это новый эксперимент, а не “починка 7a”

7b делает стандартный физический ход: замораживает вход, чтобы проверять среду. Manual anchors здесь не подаются как финальная архитектура для natural-language retrieval. Они используются как калиброванный впрыск сигнала.

Именно поэтому 7a и 7b должны читаться вместе:

- 7a локализует bottleneck automatic anchoring;
- 7b временно убирает этот bottleneck из игры.

5.3.2. Два варианта графа

Отдельно сравнивались:

- ConceptNet only;
- ConceptNet + ATOMIC.

Это было методически важно: после 7a нельзя было молча допустить, что гигантская ATOMIC shell никак не меняет topology priors.

5.3.3. `7b.1`: первая controlled-source версия

В первой версии manual cues были чистыми, но во многом еще scene-like: `check in + airport`, `home + remember`, `morning + breakfast`.

Результат: снова tie across methods.

- pairwise: 2/12;
- solved: 2/7.

Главный урок этого захода: даже ручные anchors бесполезны, если сами cues слишком общие и описывают сцену, а не механизм ответа.

5.3.4. `7b.2`: diagnostic-cue version

Во второй версии cues сделали более механизмо-ориентированными:

- `coffee + stimulant` вместо `coffee + energy`;
- `check in counter + document` вместо `check in + airport`;
- `wake up + tired` вместо `morning + breakfast`.

Это улучшило семантическую осмысленность top-k, но discrete separation все равно не появилось.

5.3.5. Результат `7b.2`

На обеих graph variants получились одинаковые агрегаты:

- primary pairwise: 1/8;
- primary solved: 1/5;
- control pairwise: 2/4;
- control solved: 1/2.

То есть controlled-source regime оказался методически правильным, но формулы снова не разошлись.

5.3.6. Что именно теперь стало ясно

После `7b` уже нельзя было объяснять стагнацию одной лишь проблемой automatic anchors. Bottleneck сместился глубже:

1. либо signal идет в broad intermediate hubs;
2. либо он попадает в graph-native adjacent manifold, а не в hand-named target node;
3. либо оба эффекта происходят одновременно.

Примеры этого были вполне содержательны:

- для morning query граф охотно выдавал `drink coffee`, а не обязательно `coffee` или `espresso`;
- для laptop query граф выдавал `power`, `battery`, `generator`-family вместо `charger`;
- для check-in query граф поднимал procedural nodes вроде `checking in` сильнее, чем `passport`.

Риск интерпретации

Здесь очень легко сделать ложный вывод “PageRank бесполезен” или “local leak не переносится”. На самом деле `7b` показал более узкую вещь: оба механизма работают слишком поздно в pipeline, если transport already lands in the wrong manifold.

5.4. `exp7c`: conjunctive cue-overlap

5.4.1. Новый вопрос

После повторяющихся tie в `7b` естественно возникла новая гипотеза: может быть, проблема не в знаменателе вообще, а в том, как складываются cue fields. Если broad hub ярко светится от одного сильного cue, а правильный ответ получает умеренную поддержку от нескольких независимых cues, то простой сумматор может систематически переоценивать первый случай.

Отсюда и возникает overlap idea:

$$\mathrm{Overlap}_{\mathrm{geom}}(v) = \left(\prod_{g} (A_g(v) + \varepsilon) \right)^{1/G}.$$

5.4.2. Что проверялось

Сравнивались:

- sum aggregation;
- sum + PageRank;
- geometric overlap;
- geometric overlap + PageRank.

То есть это уже не еще один denominator benchmark, а новая попытка изменить саму логику совместного учета нескольких cue-conditioned полей.

5.4.3. Результат

Именно здесь серия впервые довольно жестко исчерпала надежду на “еще одну формулу финального score”.

По дискретным метрикам sum-based и overlap-based варианты снова tied. В исполненном 7c это было зафиксировано как одинаковые primary outcomes на обеих graph variants; в дальнейшей постановке 8a этот итог формулируется как устойчивый tie по уровням порядка $4/10$ pairwise и $3/6$ solved для главных конкурентов.

5.4.4. Почему это содержательно важно

Overlap failure означает следующее:

> нельзя “перемножить” путь, которого вообще нет. Если $A_g(v) = 0$ для нужного cue на нужной цели, то никакая интерференция или coincidence scoring не создаст там содержательного сигнала.

Это очень сильная отрицательная находка. Она переводит разговор от финального скоринга к самой транспортной архитектуре.

5.5. Что серия 7x установила

5.5.1. Положительные результаты

1. Automatic anchoring действительно является самостоятельным bottleneck.
2. Controlled-source regime нужен и методически оправдан.
3. Эффект source echo в retrieval-списках нужно явно устранять.
4. ATOMIC shell сильно влияет на размеры и priors, но не обязана менять сами ordering outcomes.

5.5.2. Отрицательные результаты

1. На реальном эмбрионе denominator families не дают такого separation, как на золотых мирах.
2. Простая cue-overlap aggregation тоже не решает проблему.

5.5.3. Главный диагностический вывод

К концу 7x становится правдоподобной новая общая картина:

> узкое место находится не столько в пост-фактум нормировке уже пришедшего сигнала, сколько в том, что сигнал либо не доходит до правильного manifold, либо доходит не теми путями, либо приходит в слишком соседний, а не точный ответный узел.

Отсюда уже естественным образом рождается серия 8a–9a, где вопрос смещается с denominators на transport wall.

6. Серия 8a–9a: транспортная стена, similarity и достройка путей

6.1. Почему после 7x уже нельзя было просто искать еще один denominator

К началу 8a две надежды уже были достаточно честно проверены:

1. post-accumulation denominators сами по себе не отделяют методы на реальном эмбрионе;
2. overlap cue-fields тоже не создает specificity там, где transport до цели не доходит.

Следовательно, следующий вопрос должен был звучать иначе:

> может быть, проблема не в том, как пересчитать уже полученное поле, а в том, что сигнал *вообще идет не туда* или не доходит до нужного manifold?

6.2. exp8a: bypass и repair как две попытки пробить transport wall

6.2.1. Замысел

8a сознательно отвернулся от еще одной “кухни знаменателей” и взялся за два других рычага:

1. **bypass:** дать similarity прямой доступ в итоговый score, а не держать ее только как gate внутри diffusion;
2. **repair:** ослабить смазывающие типы ребер, в первую очередь `RelatedTo`, чтобы transport меньше стекал в catch-all zones.

Появилась и еще одна методологическая новация: вместо проверки одного канонического target node вводится **target manifold**. Это очень важный сдвиг. С этого места серия перестает считать провалом любой случай, когда граф выдал не exact label, а корректный соседний graph-native ответ.

6.2.2. Методы

Испытывались следующие варианты:

$$M0(v) = \frac{transport(v)}{\mathrm{PR}(v)^{\eta}},$$

$$M_{\mathrm{gate}}(v) = transport_{\mathrm{gated}}(v),$$

$$M1(v) = \frac{transport(v) + \lambda\,\mathrm{sim}(v \mid q)}{\mathrm{PR}(v)^{\eta}},$$

$$M2(v) = \frac{transport_{\mathrm{routed}}(v) + \lambda\,\mathrm{sim}(v \mid q)}{\mathrm{PR}(v)^{\eta}},$$

$$M3(v) = \frac{M2(v)}{\mathrm{genericness}(v)^{\mu}}.$$

В качестве control сохранялся и overlap-вариант из 7c.

6.2.3. Почему сравнение было честным

Особенно важно, что $M0 \to M1$ меняет только одну вещь: additive similarity. Это позволяет отделить старый semantic gate как “legacy control” от нового вопроса: помогает ли similarity, если дать ей прямой канал в score.

6.2.4. Что именно проверялось

Primary suite была достаточно строгой:

- `coffee + stimulant` против `beverage`-family;
- `check in + document` против procedural and generic distractors;
- `laptop + battery` против `office/power`;
- `wake up + tired` против `morning/bed`.

Параллельно были control queries, где broad topic не должен был быть уничтожен over-correction.

6.2.5. Главный результат

В своей окончательной, уже подправленной версии 8a дал очень чистую картину:

Метод	primary manifold	strict	mean rank	control
`M0`	2/4	1/4	232.0	2/2
`M_gate`	2/4	1/4	217.2	2/2
`M1`	2/4	1/4	63.5	2/2
`M2`	2/4	1/4	63.5	2/2
`M3`	2/4	1/4	135.8	2/2
`CTRL_overlap`	1/4	0/4	5021.2	2/2

6.2.6. Что из этого следует

Здесь одновременно есть сильный отрицательный и умеренный положительный результат.

Сильный отрицательный результат

Overlap control теперь уже не просто “не победил”, а стал явно хуже. Значит, ветка “многократное схождение cue-fields само по себе родит правильный ответ” может считаться практически исчерпанной на данном субстрате.

Умеренный положительный результат

Additive similarity действительно улучшает rank structure. Это видно по падению mean rank с 232.0 до 63.5 у $M1$. Но она не увеличивает число дискретных побед.

Иными словами, bypass способен подтягивать правильный manifold из глубины хвоста, но пока не превращается в механизм надежного переворачивания hardest pairwise cases.

Еще один отрицательный результат

Routing через `RelatedTo` в текущем виде почти ничего не прибавил: $M2$ практически совпал с $M1$. Genericness penalty в $M3$ даже ухудшила часть показателей.

6.2.7. Качественное чтение отдельных запросов

Именно top-k inspection делает выводы `8a` убедительными.

- В coffee-case `caffeine` действительно становится ближе, но над ним остаются `served hot`, `coffee shop`, `made from coffee beans`.
- В passport-case система с additive similarity не находит `passport`, а проваливается в морфологические и procedural neighbours вроде `documentor`, `photodocument`, `checking in`.
- В charger-case signal по-прежнему вязнет в `power`-family и соседнем structural noise.
- Morning-case выглядит лучше других: `drink coffee` действительно входит в сильный manifold, но не кристаллизуется как бесспорный победитель.

Вывод

`8a` не нашел “победную формулу”, но очень качественно расчистил поле. После него уже можно уверенно говорить: проблема не решается ни overlap, ни еще одним знаменателем; additive similarity полезна, но работает как мягкий rescue, а не как полноценный retrieval engine.

6.3. exp8b: de-coning similarity как проектная развилка

6.3.1. Статус

На момент этого свода 8b существует как тщательно продуманная ветка, но не как исполненный ноутбук. Это нужно проговорить прямо.

6.3.2. Почему эта ветка возникла

Как только similarity из 8a становится first-class term, встает вопрос: а хороша ли вообще сама similarity. Если embedding-space анизотропно и живет в узком конусе, то cosine margins изначально сжаты, а generic nodes становятся embedding-hubs.

Тогда часть дефекта, который мы приписываем transport, на самом деле может оказаться следствием грязной геометрии NumberBatch.

6.3.3. Что именно предполагалось проверить

8b должен был сравнить:

- raw cosine;
- mean-centering;
- all-but-the-top;
- CSLS;
- per-query z-score.

Причем сначала на intrinsic критериях (pairwise cosine distribution, hubness, margin widening), а затем как plug-in replacement для $\mathrm{sim}(v \mid q)$ в лучшем score из 8a.

6.3.4. Почему даже неисполненная ветка все равно важна

Потому что она задает очень важное ограничение для 9a: если semantic-proximity edges строить на сыром косинусе в узком конусе, можно просто переписать embedding-hubness в topology и тем самым ухудшить граф.

Открытый вопрос

Если бы 8b дал сильный положительный результат, это сделало бы de-coned similarity стандартным строительным материалом и для additive bypass, и для semantic-edge construction. Если бы не дал, это сняло бы важный конфаундер и позволило бы еще увереннее винить transport rather than cosine. Именно поэтому ветка проектно значима, даже еще не будучи прогнанной.

6.4. exp9a: dual-geometry transport и semantic-proximity edges

6.4.1. Большой вопрос

Если 8a показал, что additive similarity лишь подтягивает правильные узлы, но не решает проблему содержательно, то возникает более радикальная идея: может быть, в графе просто *нет нужных путей*, и их надо аккуратно достроить.

Отсюда и dual-geometry picture:

- graph geometry: typed relational edges;
- embedding geometry: proximity in vector space.

Новая идея состояла в том, чтобы добавить minority channel semantic edges: mutual-kNN на embeddings, но с mutual filter, degree caps и throttled conductance.

6.4.2. Постановка

Испытывались семейства:

- $G0b$: no edges, no bypass;
- $G1, G2, G3$: только semantic edges с растущим w_{sem};
- $G0a$: realistic baseline из 8a с additive similarity;
- $G1r, G2r, G3r$: edges + bypass.

Параллельно измерялись два danger checks:

- hub reflood: generic distractors не должны снова полезть наверх;
- RAG collapse: ranking не должен выродиться в pure embedding-kNN.

6.4.3. Главная сводка

Результат по основным retrieval-запросам оказался таким:

Метод	$(w_{\text{sem}}, \lambda)$	primary pairwise	median rank
G0b	(0.0, 0.0)	1/3	249
G1	(0.1, 0.0)	1/3	127
G2	(0.3, 0.0)	1/3	90
G3	(1.0, 0.0)	1/3	43
G0a	(0.0, 1.0)	1/3	111
G1r	(0.1, 1.0)	1/3	98
G2r	(0.3, 1.0)	1/3	59
G3r	(1.0, 1.0)	0/3	42

6.4.4. Как это читать

Здесь снова есть важное разделение между rank-improvement и task-solution.

- Да, semantic edges заметно подтягивают target manifolds вверх по рангу.
- Нет, они не увеличивают число дискретных побед над strongest distractor.
- При слишком сильном semantic-channel combined regime даже портится (см. $G3r$).

6.4.5. Сильная сторона 9a

Особенно интересным оказался bridge probe на alias/synonym pairs:

- `cat -> kitty`: rank 1161 $\rightarrow$ 285 $\rightarrow$ 278;
- `couch -> sofa`: 1488 $\rightarrow$ 37 $\rightarrow$ 22.

То есть там, где проблема действительно состоит в отсутствии короткого моста между очень близкими представлениями, semantic channel делает реальную работу.

6.4.6. Почему этого оказалось недостаточно

В сложных retrieval cases, особенно passport-case, semantic edges начинали соединять не “правильный ответный смысл”, а лексико-морфологическое облако: `checking in`, `check in counter`, `documentor`, `photodocument`, `redocument` и т. п.

Это очень сильный диагностический результат. Он показывает, что blind embedding-proximity repair шьет graph gaps, но шьет их не по той семантике, которая нужна ассоциативной памяти.

6.4.7. Danger checks

Здесь ноутбук тоже оказался очень полезен.

- Hub reflood не взорвался на умеренных w_{sem}: median rank generic distractors даже немного рос.
- Но RAG-collapse стал заметен: overlap top-20 с pure-sim ranking вырос примерно от 0.10 до 0.52–0.53.

Это значит, что сильный semantic channel начинает подменять граф векторным retrieval. Память при этом не столько улучшается, сколько тайком становится другим механизмом.

Вывод

9a не дал структурного “спасения” линии, но очень хорошо локализовал предел blind semantic-edge repair. Он показал, что у эмбриона действительно не хватает мостов, однако строить их как глобальные mutual-kNN

links по сырой NumberBatch-геометрии недостаточно: такие мосты тянут не только к смысловым соседям, но и к морфологическим и лексическим паразитным облакам.

6.5. Что серия 8a–9a установила

6.5.1. Отрицательные результаты

1. Overlap cue-fields не является недостающим ингредиентом retrieval.
2. Ни relation-type routing в текущем виде, ни genericness penalty не дали решающего эффекта.
3. Blind semantic-proximity edges не превращают эмбрион в хорошую память.

6.5.2. Положительные и диагностические результаты

1. Additive similarity полезна как мягкий bypass: она улучшает rank structure и поднимает цель из глубокого хвоста.
2. Semantic edges годятся как средство alias/synonym bridging.
3. Но в содержательных commonsense retrieval cases главная проблема остается глубже: signal либо идет по неправильным typed routes, либо graph-native answer organizes itself not as the named target node.

6.5.3. Почти окончательный вывод этой линии

К концу 9a становится правдоподобной уже достаточно смелая мысль:

> в текущем виде диффузионная retrieval-парадигма, даже с priors, leak laws, overlap and semantic repair, может быть уперлась в свой естественный потолок на данном субстрате.

Это не означает, что диффузия бесполезна. Это означает, что от нее, возможно, требуют того, что ближе к path-selective reasoning или к архитектуре консолидации графа, чем к простому spreading process.

7. Синтез: что доказала серия exp2–exp9a и куда она вывела

7.1. Четыре слоя выводов

Чтобы не смешивать разные типы результата, полезно собрать серию по слоям.

7.1.1. Слой 1: инфраструктура и эмбрион

Этот слой строят прежде всего exp2 и exp5.

- Реальный эмбриональный граф можно собрать и прогонять через retrieval pipeline.
- Реальные источники порождают собственный класс ошибок: URI-labeling, RelatedTo-hubness, embedding-lookup mismatch, ATOMIC fan-out.
- Write-path требует batch-aware commit semantics; наивный single-pass ломается даже на малом сценарии.

7.1.2. Слой 2: локальная механика retrieval

Этот слой задается exp3, exp4, exp6, exp6b, exp6c.

- Простой diffusion ranking действительно может предпочесть широкий хаб специфическому ответу.
- Semantic gate исправляет это лишь частично.
- Причина не сводится к плохой similarity: topology of convergence сама по себе умеет задавить более релевантный узел.
- Post-accumulation correction имеет физический смысл.
- Weighted leakage полезен, но не исчерпывает картину.
- Capacity как “эффективный объем представления” выполняет реальную работу на диагностических стендах.

7.1.3. Слой 3: глобальная структурная слава

Этот слой кристаллизуется в exp6d.

- Local response и global fame – разные механизмы.

- Мягкий PageRank prior действительно помогает, если применяется поверх локально осмысленного response.
- Полная PMI-подобная сила prior оказывается слишком агрессивной.

7.1.4. Слой 4: перенос на реальный эмбрион

Этот слой строят 7a–9a.

- Automatic anchoring – самостоятельный bottleneck.
- Controlled-source regime нужен, но даже он не возвращает separation между формулами автоматически.
- Graph-native answer manifold на реальном субстрате часто не совпадает с ручным canonical target.
- Neither denominator tweaking, nor overlap, nor blind semantic-edge repair дают решающего breakthrough.

7.2. Что именно можно считать уже опровергнутым

Серия достаточно уверенно отсекла несколько популярных, но ложных надежд.

1. **“Надо просто добавить semantic gate”** – нет, этого мало.
2. **“Проблема только в embeddings”** – нет, topology of convergence делает самостоятельную работу.
3. **“Достаточно подобрать хороший denominator”** – на реальном эмбрионе это не разделяет методы так, как на золотых мирах.
4. **“Нужно просто поощрять overlap cue-fields”** – нет, overlap не создает путь там, где его нет.
5. **“Можно достроить все мосты blind kNN edges и получить решение”** – нет, тогда память начинает подтягивать морфологическое облако и опасно съезжает к vector-RAG.

7.3. Что можно считать реально поддержанным

Здесь нужна аккуратность. Поддержанность не означает окончательную истину, но означает, что серия дала сильный положительный или диагностически устойчивый сигнал в пользу гипотезы.

1. **Typed graph matters.** Тип ребра меняет физику retrieval; `RelatedTo` нельзя считать нейтральным проводником.
2. **Post-accumulation viewpoint is correct.** Если дефект заключается в слиянии множества вкладов на broad hub, штрафовать нужно именно merged destination response.

3. **Local and global corrections are orthogonal.** Local retention и PageRank-like prior решают разные задачи.

4. **Source injection must be modeled explicitly.** На реальном графе anchoring не является технической мелочью; он архитектурно равноправен diffusion и scoring.

5. **Target manifold is the right late-stage evaluation object.** Когда графовая память возвращает соседний, но осмысленный узел, ошибочно считать это тем же типом провала, что и победу пустого хаба.

7.4. Где сейчас настоящий bottleneck

Если собрать все вместе, то bottleneck выглядит уже не как “плохая формула”. Скорее он состоит из трех связанных уровней.

7.4.1. (1) Качество графового субстрата

Эмбрион сейчас слишком смешан:

- lexical artifacts;
- morphologically noisy neighborhoods;
- неоднородные типы отношений;
- неполные мосты между реально близкими представлениями;
- слишком рыхлая интеграция episode shell.

7.4.2. (2) Качество source injection

Даже controlled-source regime понадобился не случайно. Это значит, что архитектура естественного query-to-anchor mapping еще не дошла до состояния, где последующие эксперименты можно интерпретировать без оговорок.

7.4.3. (3) Предел самого spreading-подхода

Есть все больше оснований думать, что для части задач память хочет не просто “куда растечется активация”, а что-то ближе к:

- path-sensitive retrieval;
- typed route preference;
- consolidation shortcuts;
- maybe small proof-like or script-like selection поверх поля активации.

7.5. Почему серия все же продвинулась далеко

Иногда длинная череда отрицательных и диагностических результатов производит впечатление застоя. Но в этой серии произошло обратное: исследовательский объект стал значительно яснее.

В начале речь шла почти обо всем сразу: реальный граф, embeddings, retrieval, write-path, gate, priors. К концу серии можно уже сказать заметно точнее:

- проблема не в том, что “ничего не работает”;
- проблема не сводится к одному плохо выбранному коэффициенту;
- есть рабочие локальные механизмы;
- есть рабочие диагностические стенды;
- есть честно исчерпанные ветки;
- и есть новая, уже более содержательная развилка для следующего цикла.

7.6. Что логично делать дальше

Из самой серии вытекают по меньшей мере три разумные программы продолжения.

7.6.1. Программа A: мета-аудит и смена класса retrieval

Если принять, что spreading-подход почти уперся в потолок, следующий шаг – формализовать это и перейти к path-selective retrieval. Тогда новая серия должна прямо спрашивать:

> какая причинно-смысловая траектория лучше всего объясняет запрос, а не какой узел просто ярче всех после диффузии.

7.6.2. Программа B: архитектура консолидации эмбриона

9a очень сильно намекнул, что графу не хватает внутренних мостов, но достраивать их blind kNN-способом нельзя. Значит, нужна не просто edge repair, а полноценная программа консолидации:

- alias-links;
- synonym bridges;
- shortcut links по часто используемым путям;
- controlled reverse-links;
- co-activation based strengthening;
- фильтрация морфологического мусора.

7.6.3. Программа C: развести слои памяти

Еще один естественный ход – перестать ожидать от одного плоского графа, что он одновременно будет:

- словарем;
- commonsense memory;
- episode memory;
- ассоциативной сетью синонимов и вариантов;
- рабочей средой диффузии.

Тогда следующая архитектура должна быть многослойной, а transport и retrieval – межслойными.

7.7. Финальная оценка серии

Если сформулировать общий итог одной фразой, он звучит так:

> серия exp2–exp9a не нашла окончательную retrieval law для RIN, но она очень качественно расчленила проблему и показала, где заканчивается сила нынешнего диффузионного подхода и откуда должен начинаться следующий архитектурный шаг.

Это уже большой результат. Более того, без него любой следующий шаг рисковал бы снова раствориться в хаотическом подборе коэффициентов и частных эвристик.

Вывод

Самое ценное, что дала серия, – не отдельная формула, а новая ясность. Теперь можно продолжать не с надеждой, что “еще один знаменатель все спасет”, а с довольно четким пониманием: где нам нужен другой механизм чтения, где – другой механизм роста графа, а где – более зрелая организация самой памяти как многослойной архитектуры.